

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

«Гродненский государственный университет имени Янки Купалы»

Факультет математики и информатики

Кафедра современных технологий программирования

ЖУЛЬПА УЛЬЯНА СЕРГЕЕВНА

Приложение для диагностирования распределения оперативной памяти

Курсовой проект

по дисциплине «Системное программирование»

студентки 3 курса специальности

6-05-0612-01 «Программная инженерия»

дневной формы получения образования

Научный руководитель

Куц Александр Иванович,

старший преподаватель

кафедры современных

технологий программирования

Гродно, 2025

РЕЗЮМЕ

Жульпа Ульяна Сергеевна

Тема курсового проекта

Приложение для диагностирования распределения оперативной памяти

Работа содержит: 32 страницы, 9 картинок, 13 использованных источников литературы.

Ключевые слова: диагностика памяти, оперативная память (ОЗУ), мониторинг системы, системное программирование, Windows API, WMI, процессы, потребление ресурсов, пользовательский интерфейс (UI), .NET/C#.

Цель курсового проекта – создание удобного и информативного приложения для пользователей персональных компьютеров, которое предоставляет полную, централизованную и наглядную картину распределения оперативной памяти в системе Windows, превосходя по детализации стандартные средства.

Задачи проекта: изучение принципов работы оперативной памяти в ОС Windows, анализ и сравнение возможностей встроенных и сторонних инструментов, разработка архитектуры, проектирование интерфейса и реализация работоспособного приложения.

Объект исследования – процесс диагностики и мониторинга распределения оперативной памяти в операционных системах семейства Windows. Предметом исследования являются методы и программные интерфейсы для получения и обработки информации об использовании ОЗУ системой Windows и отдельными процессами.

В проекте были использованы следующие методы: анализ, синтез, системный подход и объектно-ориентированное проектирование (ООП).

Разработанное приложение предназначено для быстрой оценки состояния оперативной памяти, выявления процессов с аномальным потреблением ресурсов и получения детальной технической информации об оперативном запоминающем устройстве (ОЗУ) компьютера. Оно служит практическим инструментом для решения повседневных задач диагностики производительности ПК.

SUMMARY

Shulpa Ulyana Sergeevna

Course project Topic

Client-Server Application for Conducting Network Quizzes

The work contains: 32 pages, 9 figures, 13 references.

Keywords: memory diagnostics, Random Access Memory (RAM), system monitoring, system programming, Windows API, WMI, processes, resource consumption, user interface (UI), .NET/C#.

The aim of the course project is to create a convenient and informative application for personal computer users, providing a complete, centralized and clear picture of Random Access Memory distribution in the Windows operating system, surpassing standard tools in detail.

Project objectives: studying the principles of Random Access Memory operation in the Windows OS, analyzing and comparing the capabilities of built-in and third-party tools, developing the architecture, designing the interface and implementing a functional application.

The object of the study is the process of diagnosing and monitoring Random Access Memory distribution in Windows family operating systems. The subject of the study is the methods and programming interfaces for obtaining and processing information about RAM usage by the Windows system and individual processes.

The following methods were used in the project: analysis, synthesis, system approach and object-oriented design (OOD).

The developed application is designed for quick assessment of the Random Access Memory state, identification of processes with abnormal resource consumption, and obtaining detailed technical information about the computer's Random Access Memory (RAM). It serves as a practical tool for solving everyday tasks of PC performance diagnostics.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	6
ГЛАВА 1	7
АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ: ИНСТРУМЕНТЫ ДИАГНОСТИКИ ОПЕРАТИВНОЙ ПАМЯТИ	7
1.1. Постановка задач для реализации приложения.....	7
1.2. Сравнительный анализ существующих решений	8
Выводы по главе 1	12
ГЛАВА 2	14
ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ДЛЯ ДИАГНОСТИКИ ОПЕРАТИВНОЙ ПАМЯТИ	14
2.1. Функциональная спецификация.....	14
2.2. Проектирование пользовательского интерфейса	17
Выводы по главе 2	20
ГЛАВА 3	21
ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ.....	21
3.1. Выбор и обоснование программных средств реализации.....	21
3.2. Реализация модулей сбора системных данных	22
3.3. Реализация ключевых функций интерфейса	24
3.4. Работа с приложением.....	25
Выводы по главе 3	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API (Application Programming Interface) – интерфейс прикладного программирования; набор готовых классов, процедур, функций и структур для взаимодействия с операционной системой или другой программой.

C# – объектно-ориентированный язык программирования, разработанный компанией Microsoft и используемый в данном проекте для создания приложения.

MVVM (Model-View-ViewModel) – архитектурный шаблон проектирования, использованный для разделения кода на три компонента: модель данных (Model), пользовательский интерфейс (View) и логику представления (ViewModel).

OOP (Object-Oriented Programming) / ООП – объектно-ориентированное программирование; парадигма программирования, основанная на использовании объектов и классов для структурирования программного кода.

OS / ОС – операционная система (Operating System).

RAM (Random Access Memory) / ОЗУ – оперативное запоминающее устройство; энергозависимая память компьютера, используемая для временного хранения данных и выполняемого кода.

WMI (Windows Management Instrumentation) – инфраструктура для управления данными и операциями в операционных системах семейства Windows. В проекте используется для получения аппаратной информации о системе.

WPF (Windows Presentation Foundation) – фреймворк для построения графического пользовательского интерфейса в приложениях под платформу .NET, выбранный для разработки в данном проекте.

Working Set (Рабочий набор) – объём физической памяти (RAM), в данный момент используемый конкретным процессом.

ВВЕДЕНИЕ

Оперативная память (ОЗУ) критически важна для производительности вычислительных систем. Эффективное распределение памяти процессами определяет отзывчивость операционной системы, стабильность приложений и возможность многозадачности. Мониторинг использования ОЗУ требует анализа на уровне процессов и потоков для диагностики утечек памяти и оптимизации производительности.

Стандартные средства Windows, такие как Диспетчер задач, предоставляют лишь общий взгляд на использование памяти, что недостаточно для полноценной диагностики. Специализированные утилиты, такие как Process Explorer и RAMMap от Sysinternals, предлагают необходимую детализацию, но имеют сложный для обычного пользователя интерфейс.

Целью данного курсового проекта является разработка приложения для диагностики распределения оперативной памяти, которое сможет предоставлять пользователям глубокий анализ памяти, а также информативный и дружелюбный интерфейс для мониторинга своего компьютера.

Для достижения этой цели необходимо:

- 1) Проанализировать управление памятью в Windows и оценить существующие программные решения.
- 2) Спроектировать архитектуру, функционал и интерфейс приложения.
- 3) Выбрать стек технологий и API для реализации.
- 4) Реализовать приложение, которое собирает, обрабатывает и визуализирует данные об использовании ОЗУ.

Конечный продукт — нативное приложение для Windows на C# с использованием WPF, имеющее структурированное представление информации, разделенное на тематические вкладки, с возможностью анализа выделенной памяти.

ГЛАВА 1

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ: ИНСТРУМЕНТЫ ДИАГНОСТИКИ ОПЕРАТИВНОЙ ПАМЯТИ

1.1. Постановка задач для реализации приложения

Существующие методы мониторинга оперативной памяти в ОС Windows имеют ряд проблем, которые существенно ограничивают эффективность диагностики для обычного пользователя.

Основная проблема — это разрозненность информации. Данные для оценки состояния памяти распределены между несколькими утилитами: общее потребление отображается в Диспетчере задач, детализация по типам системных пулов — в специализированных программах, а характеристики модулей ОЗУ — в системных сведениях. Это затрудняет оперативный анализ и требует значительных временных затрат.

Другой недостаток — отсутствие детализации на уровне процессов. Стандартные средства предоставляют ограниченные параметры (например, только «Рабочий набор»), что недостаточно для выявления источников проблем. Без доступа к информации о частных байтах, пиковом значении рабочего набора, к данным о выгружаемой и невыгружаемой памяти затруднительно отличить утечку памяти в процессе от кэширования данных системой.

На основе выявленных проблем формулируются функциональные требования к инструменту:

- 1) Объединить информацию о состоянии памяти, системных пулах и статистике по каждому процессу в одном решении.
- 2) Предоставлять пользователю расширенный набор параметров для каждого процесса, включая рабочий набор, частные байты, выгружаемый и невыгружаемый пул.
- 3) Визуализировать главные показатели (общее использование оперативной памяти и самые «объёмные» по рабочему набору процессы) с помощью графиков и/или диаграмм.
- 4) Отображать характеристики установленных модулей ОЗУ (тип, объем, частота, производитель) для возможности оценки соответствия аппаратных возможностей нагрузке.

1.2. Сравнительный анализ существующих решений

Для проектирования нового продукта необходимо оценить возможности и ограничения уже существующих решений. Для выявления лучших практик и типичных ошибок необходимо провести анализ утилит для мониторинга памяти в Windows. В данном разделе рассматриваются четыре ключевых инструмента для анализа разных уровней глубины: Диспетчер задач, Performance Monitor (perfmon.exe), Process Explorer, RAMMap.

Диспетчер задач

Диспетчер задач – это встроенная и самая известная утилита для мониторинга производительности и управления процессами в Windows, она понятна обычному пользователю и способна дать общее представление о загрузке системы и ее разных компонентов.

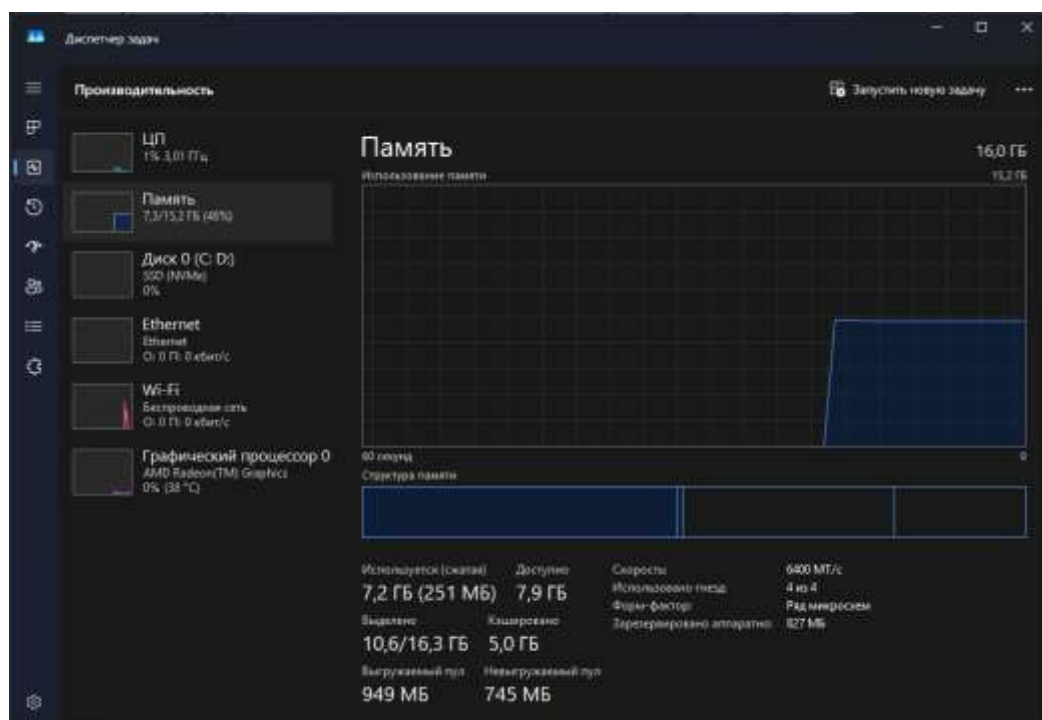


Рис. 1.1. Диспетчер задач, вкладка «Память»

Преимущества утилиты: доступность, так как это встроенная утилита (запускается по Ctrl+Shift+Esc); простой и интуитивно понятный интерфейс; графическое отображение изменения общего использования ОЗУ в реальном времени; отображение аппаратной информации; возможность увидеть потребление памяти каждым процессом, отсортировать их и завершить при необходимости.

Недостатки утилиты: указана только базовая информация о процессах, нет информации о выгружаемой и невыгружаемой памяти, частных байтах и пиковом использовании.

Performance Monitor (perfmon.exe)

Performance Monitor также является встроенным инструментом (доступен как perfmon.exe) для сбора и анализа данных системных счетчиков производительности. Используется для более глубокого анализа и создания журналов.

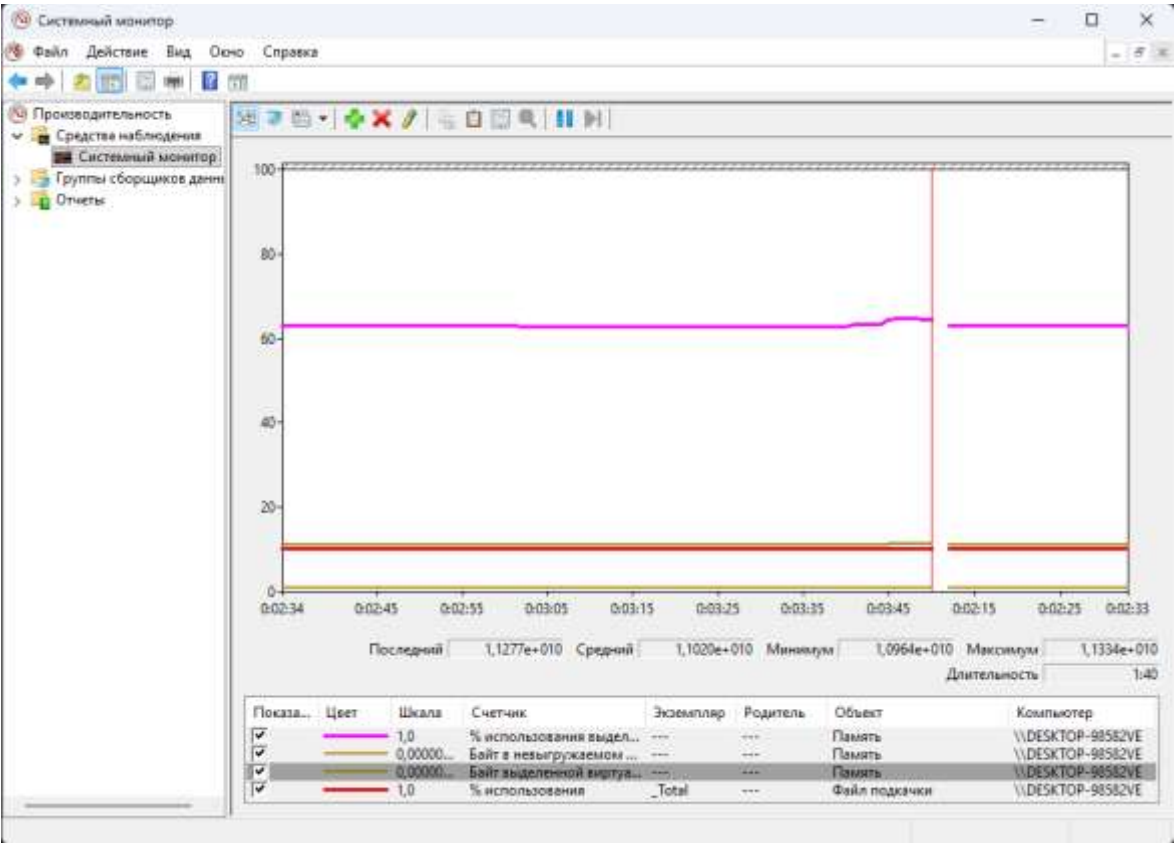


Рис. 1.2. Системный монитор, график изменения настроенных счетчиков

Преимущества утилиты: пользователю предоставлен полный доступ к счетчикам памяти, он может отслеживать десятки метрик, в том числе частные байты процесса, выгружаемый и невыгружаемый пулы. Примечательна также возможность выбора параметров для отслеживания и визуализации их изменения с течением времени.

Недостатки утилиты: интерфейс системного монитора крайне сложный для обычного пользователя, он не предназначен для быстрого мониторинга, так как требует ручного добавления нужных счетчиков. Нет единого окна, где связывались бы общая статистика памяти и детали по конкретным процессам.

Также нет никаких функций управления процессами, нельзя завершить определенный процесс или получить его свойства.

Process Explorer

Process Explorer (от Sysinternals/Microsoft) является расширенной заменой Диспетчера задач. Эта утилита способна предоставить детальную информацию о процессах, дескрипторах и загруженных DLL.

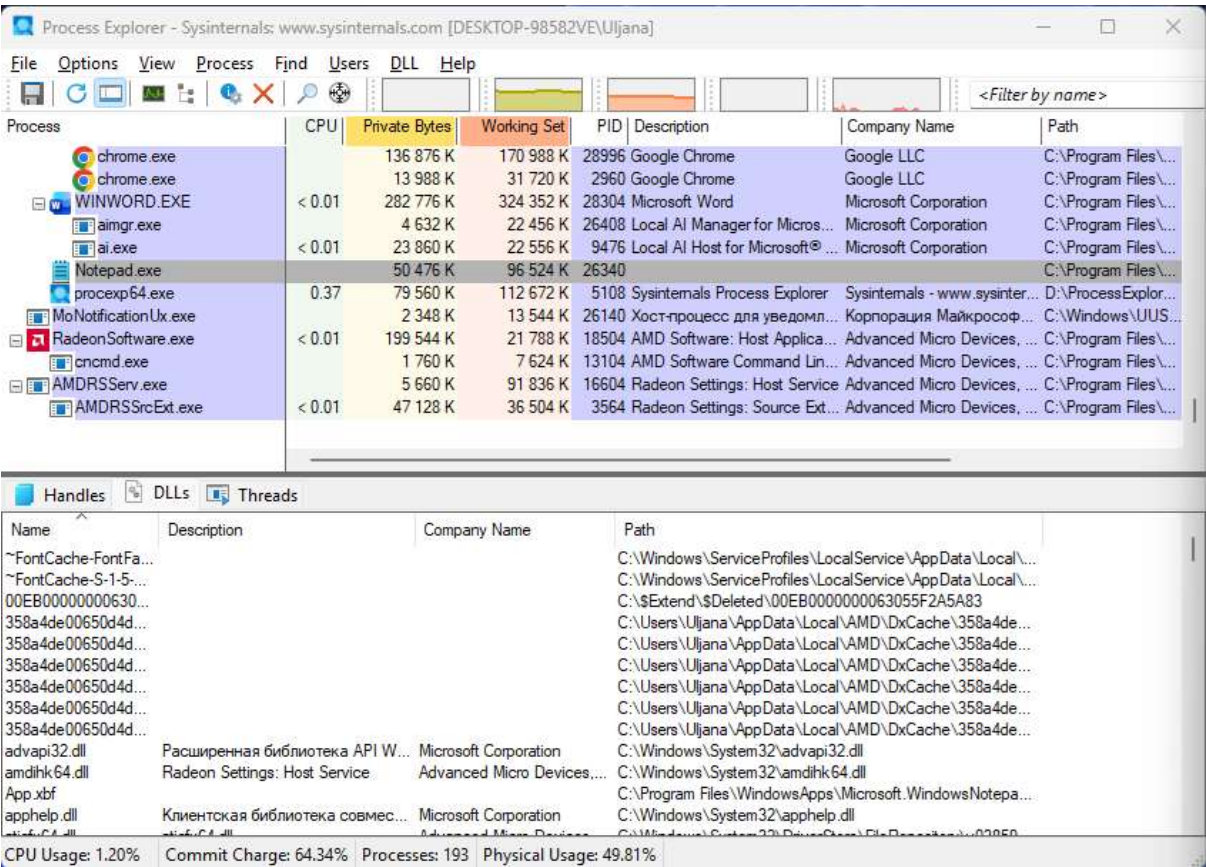


Рис. 1.3. Process Explorer, таблица процессов

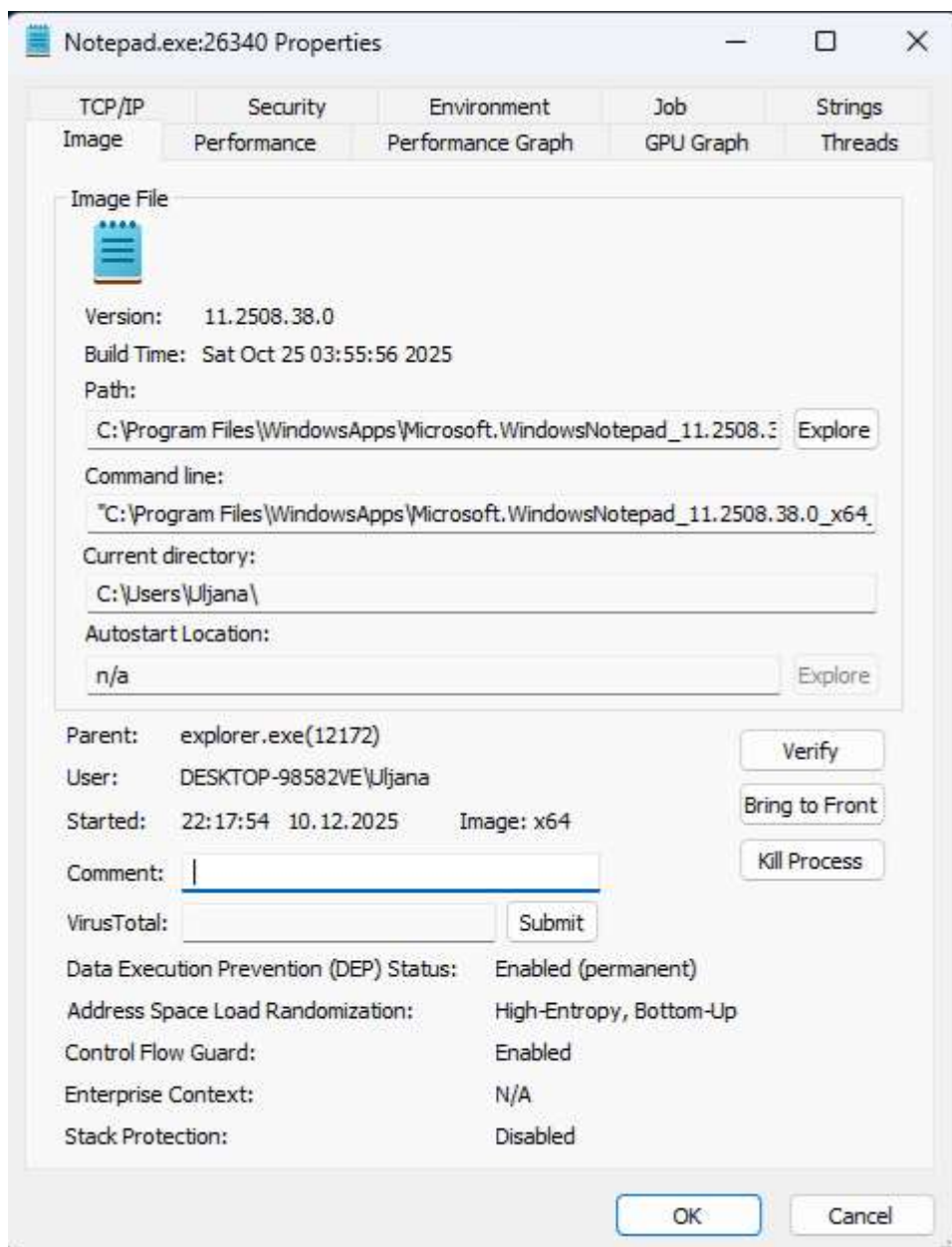


Рис. 1.4. Process Explorer, свойства процесса «Notepad.exe»

Преимущества утилиты: Process Explorer отображает намного больше столбцов, связанных с памятью (частные байты, рабочий набор, виртуальная память), а также свойства процесса. Имеется возможность выбора отображаемых столбцов. Пользователь может завершить процесс и установить ему приоритет. Эта утилита имеет более глубокую интеграцию с системой, нежели более простой и известный Диспетчер задач.

Недостатки утилиты: несмотря на возможность настройки таблицы процессов, визуализация общей картины достаточно слабая. Общее состояние системы (графики загрузки процессора, использования памяти) находятся в отдельной маленькой и незаметной области, интерфейс рассчитан на опытных

пользователей, а для обычных он слишком перегружен. Также эта утилита не предоставляет характеристики физических модулей памяти.

RAMMap

RAMMap (от Sysinternals/Microsoft) является специализированной утилитой для продвинутого анализа физического распределения оперативной памяти. Показывает, как именно ОС использует ОЗУ: здесь можно просмотреть разбивку по пулам, кэшам.

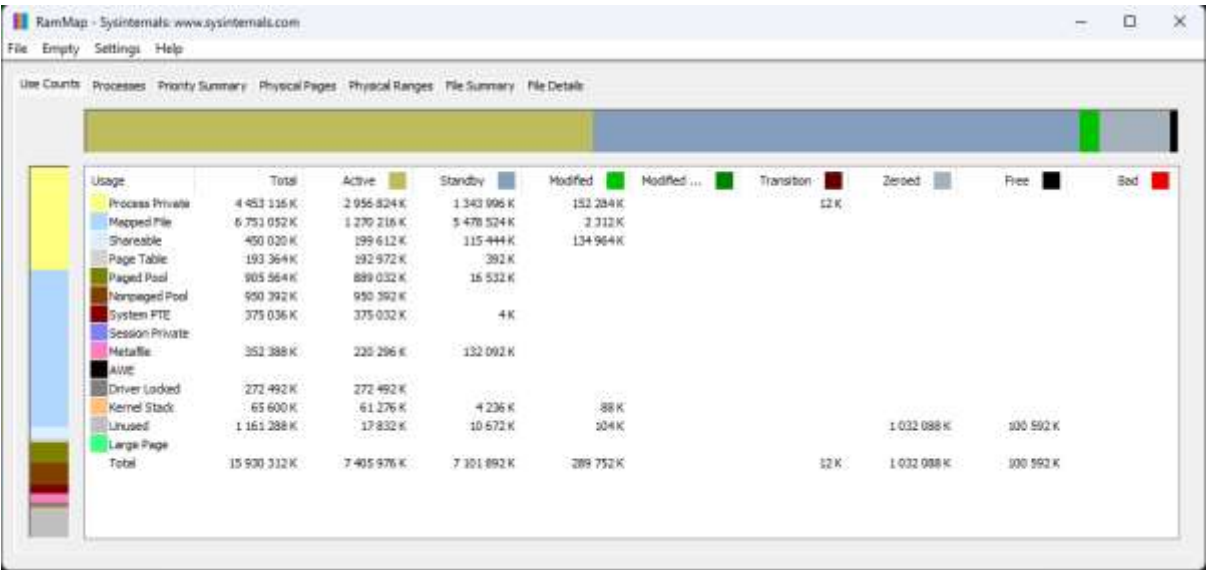


Рис. 1.5. RAMMap, вкладка «Use Counts»

Преимущества утилиты: это наиболее полный инструмент для анализа распределения оперативной памяти, позволяющий просматривать пулы, таблицы страниц, кэширование файлов. Можно просматривать распределение памяти для каждого процесса в виде удобной таблицы.

Недостатки утилиты: утилита исключительно аналитическая, в ней отсутствует управление процессами (нельзя завершать их, изучать их свойства). Приложение предназначено для опытных пользователей и разработчиков, данные представлены в виде сырых таблиц и списков, нет более дружелюбной визуализации для обычных пользователей. Также обновление данных происходит с невысокой частотой, нет динамического графика использования.

Выводы по главе 1

Проведенный анализ четырех наиболее известных утилит для диагностики оперативной памяти позволил определить проблемы в существующих решениях и конкретизировать концепцию нового инструмента.

Главным выявленным преимуществом существующих решений была возможность глубокого анализа на уровне процессов. Process Explorer и RAMMap дают доступ к более подробной информации о памяти для каждого процесса. Интерфейс диспетчера задач является самым наглядным и понятным для обычного пользователя, которому необходимо быстро оценить общую загрузку ОЗУ.

В то же время информация об оперативной памяти остается разрозненной, так как для полной диагностики памяти нужно много различных данных, которые физически находятся в совершенно разных и не связанных между собой приложениях. У многих приложений высокий порог входа или довольно узкая специализация: чем больше информации дает утилита, тем сложнее в ней ориентироваться обычному пользователю, так как для него данные без удобной визуализации будут мало чем полезны. Некоторые утилиты являются исключительно аналитическими и полностью лишены функций управления процессами, хотя наличие возможности завершить процесс или просмотреть его свойства является важным критерием для повседневного использования.

Необходимо создать инструмент, который не только объединит главные преимущества существующих решений, но и устранил выявленные недостатки:

- 1) У приложения должен быть понятный и дружелюбный для обычного пользователя интерфейс.
- 2) Необходим раздел для размещения детальной информации о процессах.
- 3) Требуется интегрировать аппаратную информацию об ОЗУ.

Таким образом, начальную концепцию приложения можно сформулировать как единое окно, которое объединяет два основных информационных блока: общее состояние и детальный анализ процессов.

В блоке общего состояния системы будут визуализированы данные об использовании ОЗУ, также здесь необходимо отобразить аппаратные характеристики модулей памяти. В блоке детального анализа процессов необходимо разместить таблицу всех процессов, а также углубленную панель с полным набором метрик памяти и свойств для выбранного процесса. Важно предусмотреть возможность завершить процесс и открыть файл, из которого он запущен.

ГЛАВА 2

ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ДЛЯ ДИАГНОСТИКИ ОПЕРАТИВНОЙ ПАМЯТИ

2.1. Функциональная спецификация

В главе 1 были определены два блока приложения: блок общего состояния системы и блок детального анализа процессов. В этом разделе будут определены функциональные требования к каждому блоку.

Блок общего состояния системы предназначен для быстрой оценки общего состояния оперативной памяти, для чего будут использованы средства визуального отображения, и аппаратной среды. На этой вкладке определим независимые блоки: распределение оперативной памяти, график использования оперативной памяти, аппаратная информация и диаграмма «Топ-5 процессов по памяти».

Распределение оперативной памяти

В данном блоке необходимо отображать сводные числовые показатели, изменяющиеся в реальном времени, а именно:

- 1) Используемая память – это объем физической памяти, занятый в данный момент.
- 2) Доступная память – это объем физической памяти, готовый к немедленному использованию процессами.
- 3) Выделенная память – это общий объем виртуальной памяти, который гарантирован системой для всех процессов (состоит из физической памяти и файла подкачки).
- 4) Кэшированная память – это объем памяти, используемый для кэширования файлов и данных системы.
- 5) Выгружаемый пул – это объем выделенной ядру памяти, которая может быть выгружена в файл подкачки.
- 6) Невыгружаемый пул – это объем критической памяти ядра, которая всегда должна находиться в оперативной памяти и не может быть выгружена в файл подкачки.

Все указанные показатели достаточны для быстрой оценки распределения памяти в системе, они должны обновляться с высокой частотой (по умолчанию 1,5 секунды)

График использования оперативной памяти

Используемая память является важным параметром оценки, поэтому необходимо визуально отображать динамику изменения этого показателя. Наиболее удобным форматом для представления является график за последние 60 секунд, обновление происходит синхронно с обновлением числовых показателей (с частотой 1,5 сек.), таким образом, исторические данные хранятся в буфере на 40 значений. График должен иметь метки осей (время, объем в максимальной величине) и сетку для удобства чтения.

Аппаратная информация

В данном блоке должна находиться постоянная и единовременно извлекаемая при запуске приложения информация о физических характеристиках установленных модулей ОЗУ, а именно:

- 1) Скорость – эффективная частота передачи данных (например, «3200 МТ/с»).
- 2) Тип памяти – технология памяти (например, «DDR4»).
- 3) Производитель – название компании-производителя модуля.
- 4) Использование гнезд – формат «X из Y», где X – количество занятых слотов, а Y – общее количество слотов на материнской плате.
- 5) Форм-фактор – физический стандарт (например, «SODIMM»).
- 6) Серийный номер – уникальный идентификатор модуля.
- 7) Зарезервированная аппаратно память – объем памяти, зарезервированный для системных нужд.

Топ-5 процессов по памяти

Для быстрого мониторинга состояния памяти будет полезно отображение самых "объемных" по памяти процессов, здесь необходимо использовать диаграмму для отображения доли потребления рабочего набора пятью процессами с наибольшим значением этого показателя. Под диаграммой необходимо расположить таблицу-легенду для однозначного сопоставления сегмента диаграммы с процессом. Для каждого процесса указываются ID (идентификатор процесса), имя (название исполняемого файла) и рабочий набор (значение в оптимальной по размеру величине). Данные обновляются синхронно с остальными блоками на вкладке – каждые 1,5 сек и в таблице-легенде отображаются в сортировке по убыванию значения рабочего набора.

Блок детального анализа процессов предназначен для просмотра списка всех выполняющихся в системе процессов, включая системные и пользовательские.

На этой вкладке должны находиться таблица со всеми процессами системы, а также окна для детальной информации о выбранном в таблице процессе.

Таблица всех процессов

Функциональность таблицы предусматривает возможность сортировки по любой ее колонке по возрастанию и убыванию, обновление с высокой частотой (каждые 1,5 сек.), возможность завершить процесс.

Для удобства использования в таблице определены лишь основные колонки:

- 1) Значок исполняемого файла процесса.
- 2) PID – числовой идентификатор процесса.
- 3) Название процесса (имя исполняемого файла без пути).
- 4) Продолжительность работы процесса в формате ЧЧ:ММ:СС.
- 5) Текущий размер виртуальной памяти процесса в байтах.
- 6) Текущий размер рабочего набора процесса в байтах.
- 7) Действие «Завершить» для осуществления попытки принудительного завершения соответствующего процесса.

Пользователь может выбрать строку с определенным процессом, кликнув на нее. При условии наличия выбранного процесса на вкладке отображаются две логические области с детальной информацией о процессе: блок использования оперативной памяти и блок системных свойств процесса.

Использование оперативной памяти выбранным в таблице процессом

В данной области необходимо отобразить значения расширенных счетчиков памяти для выбранного процесса. Каждый показатель выводится в двух форматах: точное значение в байтах и автоматически отформатированное значение в удобочитаемом виде (КБ, МБ и т.д.). Список показателей:

- 1) Объем рабочего набора.
- 2) Пиковое значение рабочего набора.
- 3) Частные байты.
- 4) Виртуальная память.
- 5) Выгружаемая память.
- 6) Невыгружаемая память.

Свойства выбранного в таблице процесса

В данной области необходимо отобразить общую и административную информацию о выбранном процессе. Список свойств:

- 1) Суммарное время использования центрального процессора (ЦП) процессом, формат – ЧЧ:ММ:СС.
- 2) Дата и время запуска процесса, формат – ММ/ДД/ГГГГ ЧЧ:ММ:СС АМ/РМ.
- 3) Имя пользователя или учетной записи, от имени которой выполняется процесс.
- 4) Полный путь к исполняемому файлу. Здесь же необходимо реализовать интерактивную возможность открыть папку, содержащую указанный файл.
- 5) Числовое значение приоритета планирования потока.
- 6) Текущее количество потоков, выполняющихся в контексте процесса.

2.2. Проектирование пользовательского интерфейса

Основные принципы проектирования пользовательского интерфейса — это интуитивность, минимализм и фокус на данных. Интерфейс должен быть построен по образцу стандартных системных утилит Windows: навигация будет реализована по принципу одной всегда доступной области с переключаемыми вкладками, где разделяются два режима работы: общий обзор и детальный анализ процессов.

Интерфейс приложения строится вокруг двух основных вкладок, доступных через верхнюю панель навигации: «Главная» и «Монитор процессов». При запуске приложения пользователь по умолчанию попадает на вкладку «Главная», которая предназначена для быстрой оценки общего состояния оперативной памяти системы. Макет данной вкладки разработан с учетом принципа информационной иерархии: наиболее важные и обобщенные данные размещены вверху и слева, а вспомогательные и детализированные — справа и внизу. Рабочая область разделена на три четко обозначенных блока: «Распределение ОП», «Аппаратная информация» и «Топ-5 процессов». Для визуализации динамики использования памяти применен линейный график, а для отображения распределения памяти между процессами — круговая диаграмма с таблицей-легендой.

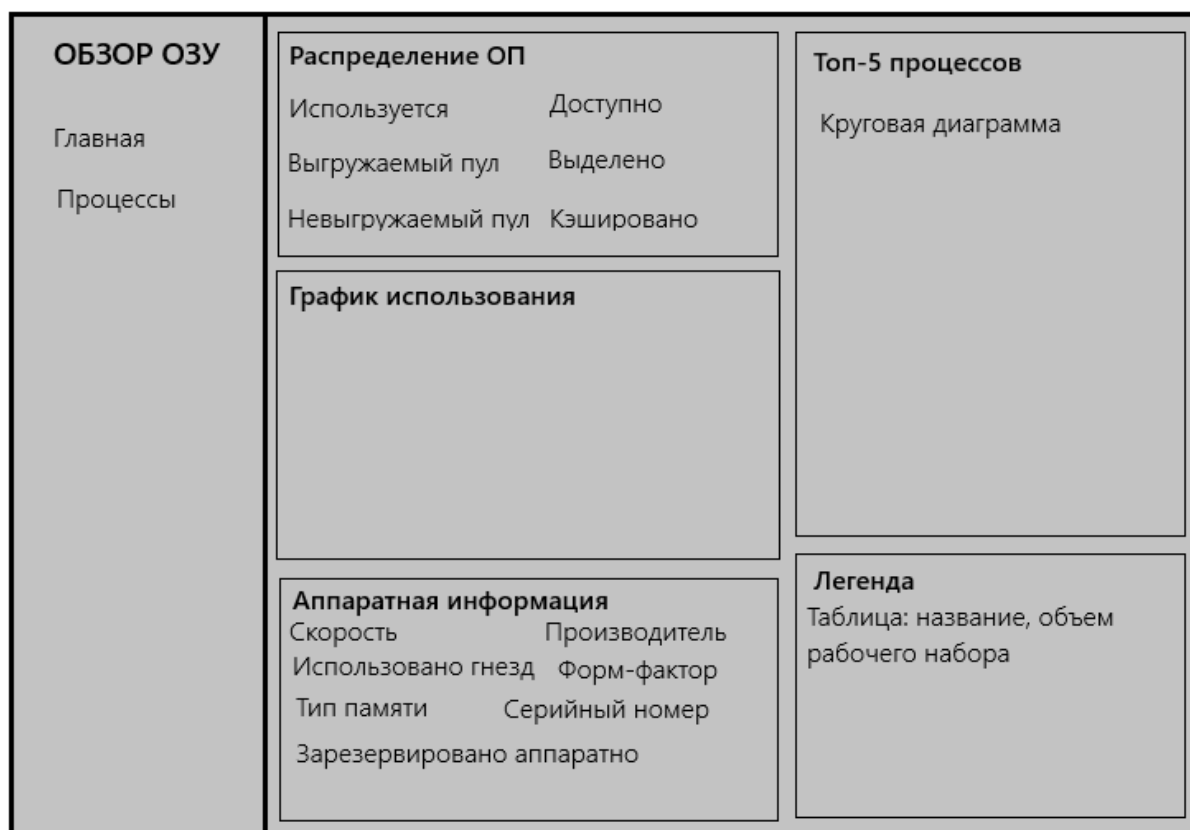


Рис. 2.1. Макет главной вкладки

Главная вкладка («Главная») спроектирована как информационная панель (dashboard). Ее цель — предоставить пользователю моментальный снимок состояния оперативной памяти. В верхней части расположен блок «Распределение ОП», содержащий ключевые числовые метрики: общий объем физической памяти, используемый и доступный объем, размер выгружаемого и невыгружаемого пула, а также объем кэшированной памяти. Непосредственно под цифровыми показателями размещен график «Использования ОЗУ», который в реальном времени отображает изменение нагрузки за последние 60 секунд, позволяя быстро оценить тренд. В левой нижней части вкладки находится блок «Аппаратная информация», где статическими текстовыми полями выводятся характеристики установленных модулей памяти: тип, скорость, производитель, форм-фактор, серийный номер, количество занятых и свободных слотов, а также объем памяти, зарезервированной аппаратно. В правой части вкладки расположен интерактивный блок «Топ-5 процессов», состоящий из круговой диаграммы, визуализирующей долю памяти, потребляемую каждым из пяти самых «прожорливых» процессов, и таблицы-легенды, которая расшифровывает эти доли, отображая PID, имя процесса и точный объем его рабочего набора в байтах.

Переход к детальному анализу осуществляется интуитивно — через нажатие на вторую вкладку в верхней панели навигации, подписанную «Монитор процессов». Такой подход соответствует шаблону проектирования «мастер-детализация», где на первой вкладке представлен общий контекст, а на второй — детализированные данные и инструменты для управления.

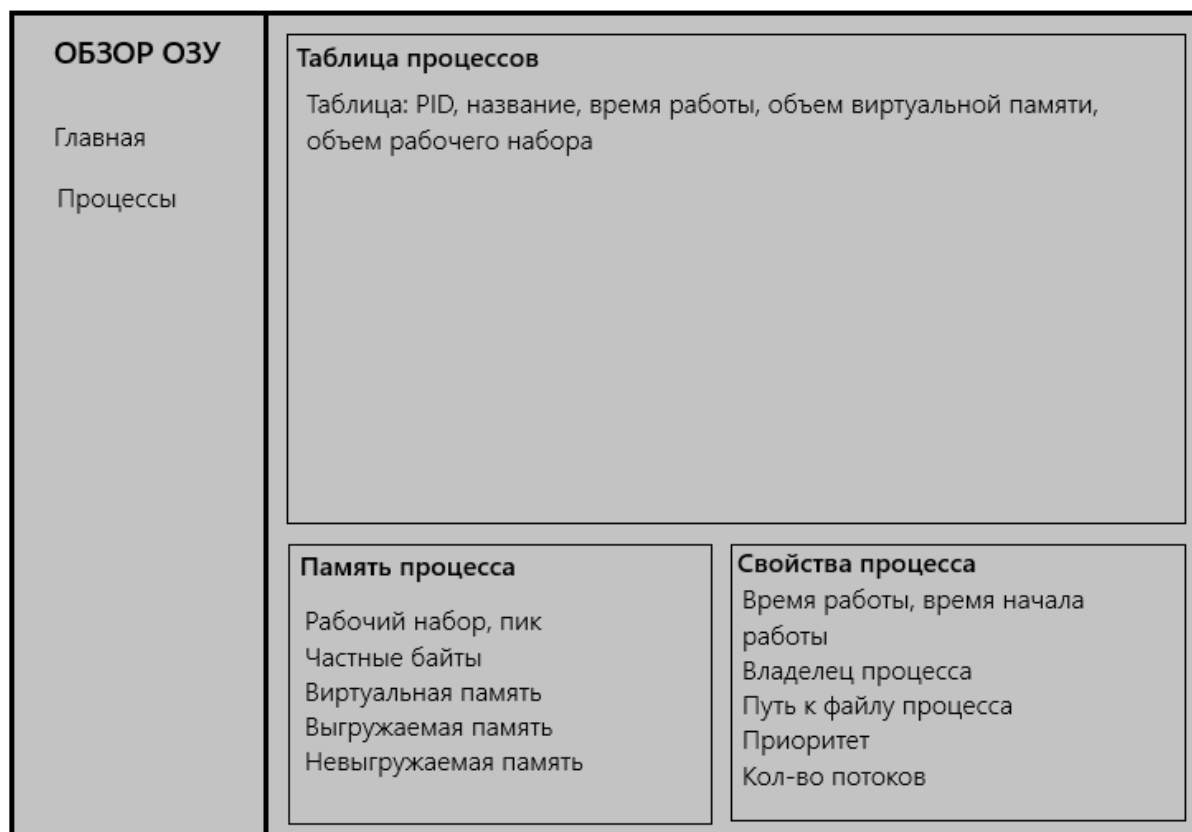


Рис. 2.2. Макет вкладки с таблицей процессов

Вкладка «Монитор процессов» спроектирована для глубокого анализа и управления процессами системы. Ее интерфейс организован по классической схеме «список-детализация». Верхнюю и основную часть окна занимает таблица всех системных процессов. Для каждого процесса в строке таблицы отображаются: иконка приложения, идентификатор (PID), название, время работы, объем виртуальной памяти и объем рабочего набора. Таблица поддерживает сортировку по любому столбцу (как по возрастанию, так и по убыванию), а также содержит в каждой строке кнопку «Завершить» для немедленного завершения выбранного процесса.

При выборе любой строки в таблице, нижняя часть интерфейса автоматически заполняется детальной информацией о выбранном процессе. Эта область разделена на две логические панели: «Память процесса» и «Свойства процесса».

В панели «Память процесса» в виде пар «название метрики — значение» представлены все ключевые счетчики потребления ОЗУ: рабочий набор, пик рабочего набора, частные байты, объем виртуальной памяти, выгружаемая и невыгружаемая память. Каждое значение дублируется в двух форматах — точное число в байтах и удобочитаемая строка (например, «1.5 ГБ»).

Панель «Свойства процесса» содержит системную информацию: время работы и время начала, имя владельца процесса, полный путь к исполняемому файлу (с активной ссылкой для открытия в проводнике), базовый приоритет и количество потоков.

Выводы по главе 2

В результате проектирования была полностью определена и детализирована структура разрабатываемого приложения.

Разработаны функциональные требования, определяющие поведение системы. Режим работы приложения разделен между двумя вкладками: «Главная» с общим состоянием системы и «Монитор процессов» для детального анализа исполняемых в системе процессов.

Логическая архитектура спроектирована на принципе разделения ответственности между слоями для низкой связанности компонентов. Определены три основных модуля (сбора данных, обработки, представления) и описан поток данных между ними.

Созданы макеты пользовательского интерфейса, соответствующие принципам интуитивности и минимализма. Интерфейс решает ключевую проблему разрозненности информации, выявленную в ходе анализа, путем централизации данных в двух логически связанных вкладках.

Все ключевые проектные решения приняты, функциональные и нефункциональные требования формализованы. Структура данных, алгоритмы взаимодействия компонентов и визуальное представление полностью определены.

ГЛАВА 3

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

3.1. Выбор и обоснование программных средств реализации

Для разработки приложения выбран язык программирования C# и платформа .NET. Они глубоко интегрированы с операционной системой Windows, имеют обширную документацию и прямой доступ ко всем необходимым системным интерфейсам (API). Библиотеки .NET, такие как System.Diagnostics, предоставляют готовые средства для работы с процессами.

Пользовательский интерфейс реализован с использованием фреймворка Windows Presentation Foundation (WPF). WPF был выбран благодаря своей системе привязки данных (Data Binding), которая позволяет эффективно связывать визуальные элементы, описанные в разметке XAML, с бизнес-логикой на C#. Такой подход напрямую поддерживает архитектурный паттерн Model-View-ViewModel (MVVM), обеспечивающий четкое разделение кода интерфейса и внутренней логики приложения. Для практической реализации MVVM и сокращения шаблонного кода использовалась библиотека CommunityToolkit.Mvvm, предоставляющая удобные классы (ObservableObject) и атрибуты для генерации команд (RelayCommand).

Сбор системной информации организован с применением нескольких стандартных для Windows технологий.

Для получения аппаратных характеристик модулей ОЗУ используется технология WMI (Windows Management Instrumentation) через пространство имен System.Management и запросы к классам Win32_PhysicalMemory и Win32_PhysicalMemoryArray.

Для мониторинга динамических показателей использования памяти в реальном времени (общий объем, пулы, кэш) применяются счетчики производительности Windows (Performance Counters) через класс PerformanceCounter.

Список процессов и их основные свойства получаются с помощью класса System.Diagnostics.Process. Для извлечения дополнительных данных, таких как владелец процесса, используется комбинация WMI (Win32_Process) и нативных вызовов Windows API. Также приложение просматривает издателя сертификата

каждого процесса при помощи функций из пространства имен `System.Security.Cryptography.X509Certificates`.

Для создания графиков и диаграмм был выбран LiveCharts2 — современная библиотека для визуализации данных.

Выбранный технологический стек оптимален для данного проекта, так как обеспечивает высокую производительность, полный доступ к низкоуровневой информации и позволяет создать современный интерфейс.

3.2. Реализация модулей сбора системных данных

Архитектура приложения построена вокруг трех независимых сервисных модулей, которые создаются один раз при запуске главного окна (MainWindow) и включаются в соответствующие ViewModel. Имеются следующие модули: модуль для сбора аппаратных характеристик ОЗУ, модуль мониторинга текущего распределения оперативной памяти, модуль для мониторинга процессов.

Модуль сбора аппаратных характеристик ОЗУ представлен классом `MemoryHardwareService`. Данный сервис необходим для однократного сбора информации о физических модулях оперативной памяти при запуске приложения. Источником данных является технология WMI.

При создании сервиса выполняются два запроса: к классу `Win32_PhysicalMemory` для получения данных о каждом установленном модуле (объем, скорость, производитель, серийный номер, тип памяти в виде SMBIOS-кода, форм фактор) и к классу `Win32_PhysicalMemoryArray` для определения общего количества доступных и использованных слотов на материнской плате.

Полученные SMBIOS-коды преобразуются в читаемый текст. Затем вычисляются агрегированные показатели: общий объем памяти, средняя скорость, обобщенный тип и форм-фактор. Также в этом сервисе осуществляется запрос к классу `Win32_OperatingSystem` для определения зарезервированного аппаратно объема памяти.

В результате формируется объект `MemoryHardwareInfo`, который передается в `PerformanceViewModel` и отображается в блоке аппаратной информации на главной вкладке.

Модуль мониторинга распределения памяти представлен классом `MemoryAllocationInfoService`. Данный сервис необходим для циклического сбора актуальных данных об использовании оперативной памяти в режиме

реального времени. Источниками данных являются комбинация технологии WMI и счетчиков производительности Windows (Performance Counters).

При работе сервис выполняет следующие действия в цикле с заданным интервалом (1.5 секунды). Сначала через WMI-запрос к классу Win32_OperatingSystem получаются базовые метрики: общий физический объем, свободная физическая память и данные о виртуальной памяти. Затем считываются высокочастотные счетчики производительности, включая Available Bytes, Pool Paged Bytes, Pool Nonpaged Bytes, Committed Bytes и группу счетчиков Cache Bytes для точного расчета кэшированных данных.

На основе полученных величин вычисляются производные показатели. Ключевой метрикой «Используется» (Used) является разница между общим физическим объемом и доступной памятью (Available Bytes).

В результате формируется снимок состояния MemoryAllocationInfo. Этот объект публикуется сервисом, на что подписан PerformanceViewModel, что приводит к автоматическому обновлению всех связанных элементов интерфейса: числовых показателей, графика загрузки и данных для диаграммы топ-процессов на вкладке «Обзор системы».

Модуль мониторинга процессов представлен классом ProcessMonitorService. Данный сервис необходим для периодического сбора информации о всех запущенных в системе процессах. Источниками данных выступают API .NET (System.Diagnostics.Process), вызовы WinAPI и анализ файлов.

При работе сервис в цикле обновления получает базовый список процессов через Process.GetProcesses(). Для каждого процесса извлекаются стандартные свойства: идентификатор, имя, рабочий набор, частные байты, пиковое использование, виртуальная и пул-память, приоритет, время запуска, количество потоков и дескрипторов. На основе времени запуска рассчитывается длительность работы.

Затем сервис получает дополнительные свойства процесса. Путь к исполняемому файлу извлекается из MainModule.FileName. Владелец процесса определяется через вызовы WinAPI (OpenProcessToken, GetTokenInformation) для получения и преобразования SID. Цифровая подпись файла проверяется через X509Certificate2.CreateFromSignedFile() для получения имени издателя. Иконка процесса извлекается с помощью Icon.ExtractAssociatedIcon().

В результате для каждого процесса формируется объект ProcessModel. Полная обновленная коллекция этих моделей передается на поток UI. Она используется

в `ProcessMonitorViewModel` для заполнения таблицы и реализации команд, а также в `PerformanceViewModel` для быстрого вычисления и отображения топ-5 процессов по потреблению памяти.

3.3. Реализация ключевых функций интерфейса

Чтобы обеспечить корректную работу приложения и удобство для пользователя, реализованы важные системные функции для обновления данных в реальном времени, автоматическое форматирование отображаемых величин и управление процессами.

Обновление данных в реальном времени обеспечивается таймерами, которые встроены в сервисные классы `MemoryAllocationInfoService` и `ProcessMonitorService`. В этих сервисах создаются таймеры (`System.Threading.Timer`), которые каждые 1.5 секунды иницируют сбор новых данных. После получения новых данных сервисы используют `Application.Current.Dispatcher`, чтобы изменения свойств произошли в потоке пользовательского интерфейса.

Для представления числовых значений в удобочитаемом виде реализована система конвертации. Так как сервисы получают информацию о памяти в байтах, необходим класс для их конвертации в большие величины, удобные для восприятия. Для этого реализован класс `MemoryUnitConverter`, который переводит большое число (байты) в строки с автоматическим подбором единиц (КБ, МБ, ГБ, ТБ) и округлением. Этот конвертер используется в свойствах-геттерах моделей данных, которые напрямую привязываются к элементам интерфейса.

Приложение допускает возможность управления процессами в виде завершения процесса и открытия расположения его файла. Эти функции реализованы в `ProcessMonitorViewModel` с помощью команд.

Команда открытия расположения файла при клике на соответствующую кнопку проверяет доступность пути к исполняемому файлу выбранного процесса. Если он существует, запускается системный процесс `explorer.exe` с параметром `/select`, в результате открывается проводник с выделенным файлом. В случае ошибки пользователю выводится понятное предупреждение.

Команда завершения процесса находит выбранный процесс в коллекции по PID. Перед выполнением запрашивается подтверждение у пользователя, выполняется проверка имени процесса на соответствие списку важных системных процессов (`svchost`, `winlogon`, `csrss`, `lsass` и др.). После подтверждения

вызывается метод `Process.Kill()` для принудительного завершения. После успешного завершения процесс автоматически исчезает из таблицы при следующем цикле обновления данных.

3.4. Работа с приложением

Приложение реализует интуитивно понятный сценарий работы, позволяющий пользователю от просмотра общего состояния системы перейти к детальному анализу процессов системы. Взаимодействие строится вокруг двух вкладок: «Главная» и «Монитор процессов».

После запуска приложение отображает на вкладке «Главная» сводку о состоянии оперативной памяти, аппаратных характеристиках ОЗУ для оценки общей ситуации.

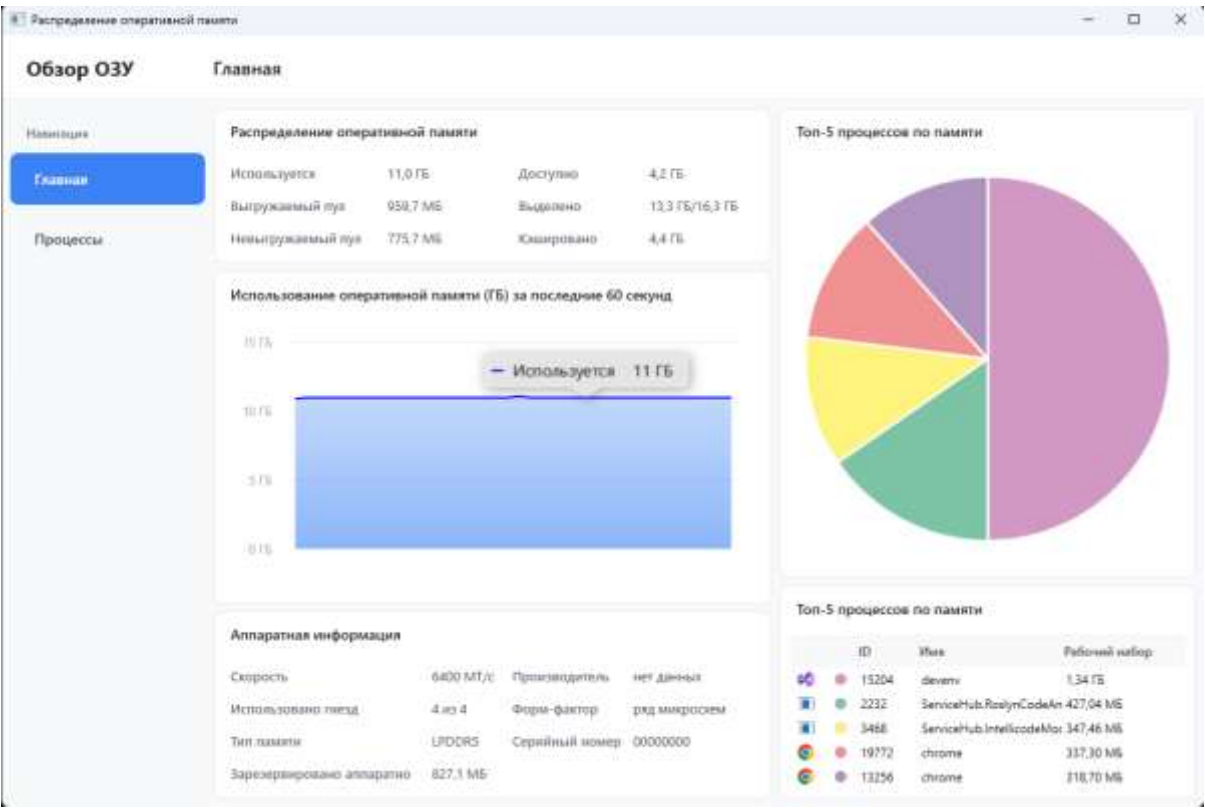


Рис. 3.1. Обзор ОЗУ, вкладка «Главная»

Рабочая область вкладки разделена на три информационных блока: распределение оперативной памяти, аппаратная информация, топ-5 процессов по объему рабочего набора.

В верхней части окна представлены числовые показатели: объем физической памяти, объем используемой и доступной памяти, размер выделенной памяти.

Ниже расположен линейный график, отображающий динамику изменения показателя «Используется» за последние 60 секунд.

В блоке аппаратной информации (нижняя часть окна) отображаются статические характеристики установленных модулей ОЗУ: тип памяти, скорость, производитель, форм-фактор, серийный номер, количество используемых и доступных слотов на материнской плате. Также выводится объем зарезервированной аппаратно памяти.

В блоке «Топ-5 процессов по памяти», который расположен в правой части вкладки, представлена круговая диаграмма, которая показывает пропорциональное распределение памяти между пятью процессами с наибольшим рабочим набором. Под диаграммой находится таблица-легенда с расшифровкой: для каждого процесса указаны PID, имя и точный объем рабочего набора.

Таким образом, с помощью всего одной вкладки пользователь узнает общий объем и текущую загрузку памяти, аппаратные характеристики ОЗУ. Для анализа отдельных процессов необходимо перейти на вкладку «Монитор процессов», которая предоставляет полный контроль над списком всех выполняемых в системе программ и детальную информацию о каждой из них.

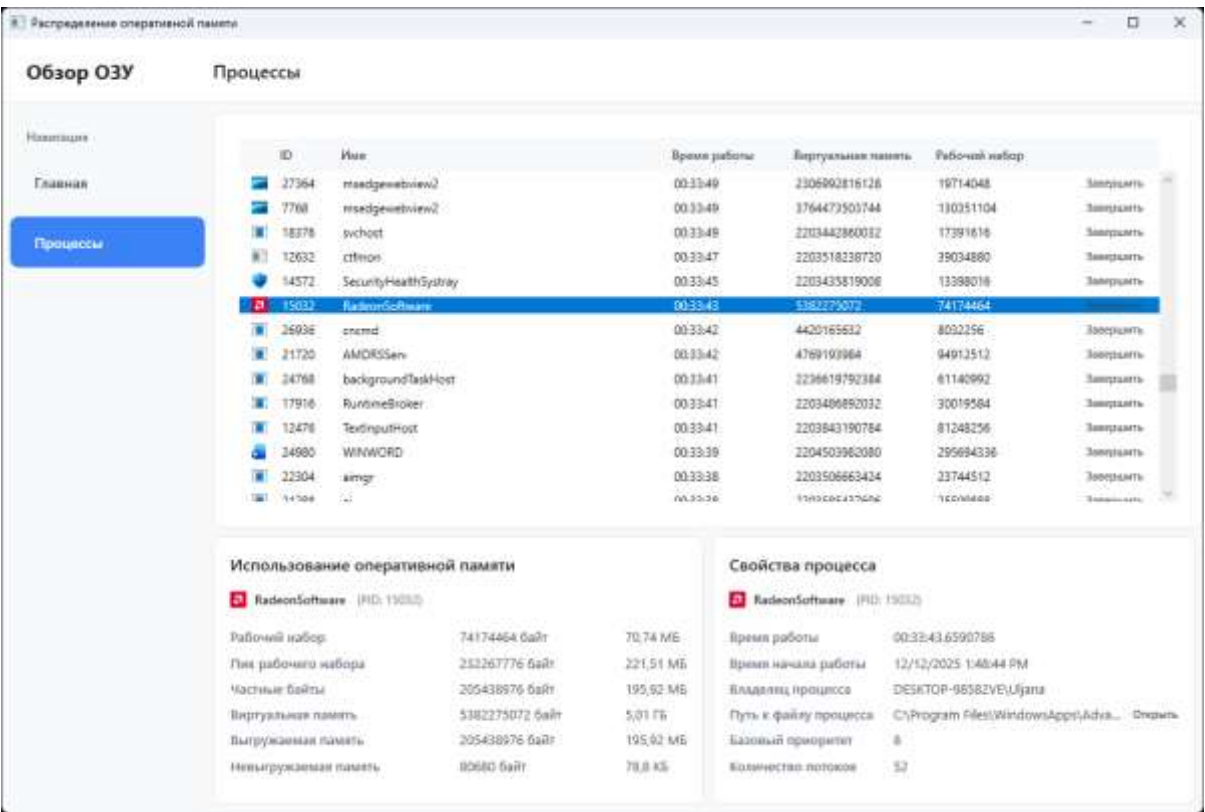


Рис. 3.2. Обзор ОЗУ, вкладка «Монитор процессов»

Интерфейс организован по схеме "список-детализация". В верхней части расположена таблица с полным перечнем всех процессов системы. Для каждого процесса отображаются столбцы: иконка, PID, имя, время работы, объем виртуальной памяти и рабочего набора в байтах. Пользователь может сортировать таблицу по любому столбцу (по возрастанию и убыванию), также в каждой строке присутствует кнопка завершения процесса.

В нижней части расположены панели детальной информации о выбранном процессе. При выборе строки эти панели заполняются данными, разделенными на две группы: использование оперативной памяти и свойства процесса. В первой группе представлены все счетчики памяти для процесса: рабочий набор, пик рабочего набора, частные байты, виртуальная память, выгружаемая и невыгружаемая память. Каждое значение дублируется в двух форматах: точное число в байтах и удобочитаемая величина. В группе свойств процесса содержится системная информация о времени работы и времени начала, владельце, пути к исполняемому файлу с возможностью его открытия в проводнике, базовый приоритет и количество потоков.

Выводы по главе 3

В третьей главе была осуществлена практическая реализация проекта приложения для диагностики оперативной памяти. На первом этапе был определен выбор технологического стека разработки. В качестве фундаментальной платформы были выбраны язык программирования C# и среда выполнения .NET, что обеспечило глубокую интеграцию с операционной системой Windows и предоставило прямой доступ к необходимым системным интерфейсам (API). Для создания пользовательского интерфейса был применен фреймворк Windows Presentation Foundation (WPF), который благодаря мощной системе привязки данных позволил четко разделить визуальное представление (XAML) и бизнес-логику (C#). Это разделение было формализовано с помощью архитектурного паттерна Model-View-ViewModel (MVVM), реализация которого была значительно упрощена за счет использования библиотеки CommunityToolkit.Mvvm.

Далее в главе подробно описана архитектура и принцип работы трех ключевых сервисных модулей, составляющих ядро функциональности приложения. Модуль MemoryHardwareService отвечает за однократный сбор статической аппаратной информации о модулях оперативной памяти при помощи запросов к технологии WMI. Модуль MemoryAllocationInfoService реализует циклический мониторинг динамических показателей использования

памяти в режиме реального времени, комбинируя данные от WMI и высокочастотных счетчиков производительности Windows. Модуль ProcessMonitorService осуществляет периодический сбор детальной информации обо всех системных процессах, используя как стандартные классы .NET, так и низкоуровневые вызовы WinAPI для получения таких данных, как владелец процесса и цифровая подпись его файла.

Были успешно реализованы все спроектированные функции интерфейса. Для удобства восприятия числовых данных реализован механизм автоматического форматирования, преобразующий значения в байтах в удобочитаемые строки с подходящими единицами измерения. В программе присутствует возможность управления процессами в виде принудительного завершения с подтверждением и открытия расположения исполняемого файла в проводнике Windows.

Все запланированные компоненты были успешно реализованы, в результате создан целостный программный продукт. Итоговое приложение предоставляет пользователю последовательный и удобный опыт работы: от быстрой оценки общего состояния системы и аппаратных характеристик на вкладке «Главная» до глубокого детального анализа и управления отдельными процессами на вкладке «Монитор процессов». Разработанный инструмент полностью соответствует поставленной цели, предлагая комплексное решение для эффективной диагностики распределения оперативной памяти в операционной системе Windows.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно решена задача разработки приложения для диагностики распределения оперативной памяти в ОС Windows.

На первом этапе был проведен детальный анализ проблемы мониторинга памяти и существующих инструментов. Исследование показало, что стандартные средства (Диспетчер задач) обладают недостаточной детализацией, а профессиональные утилиты (Process Explorer, RAMMap) имеют интерфейс, сложный для рядового пользователя. На основе этого были сформулированы ключевые требования к новому продукту: централизация информации, глубина анализа и интуитивный интерфейс.

Далее был выполнен этап проектирования, в результате которого определилась четкая архитектура приложения, основанная на паттерне MVVM, и были разработаны макеты двух основных вкладок: «Обзор системы» для общего мониторинга и «Монитор процессов» для детального анализа.

В рамках программной реализации был выбран и применен технологический стек C#/.NET и WPF, оптимально подходящий для создания нативных Windows-приложений. Были разработаны три независимых сервисных модуля (MemoryHardwareService, MemoryAllocationInfoService, ProcessMonitorService), которые обеспечивают сбор аппаратных данных, мониторинг распределения памяти в реальном времени и получение расширенной информации о процессах соответственно.

Цель проекта достигнута: создано работоспособное приложение, которое объединяет удобство мониторинга, характерное для встроенных средств Windows, с глубиной диагностики, присущей профессиональным утилитам. Пользователь получает в одном интерфейсе полную картину состояния оперативной памяти — от общих показателей и аппаратных характеристик до детального анализа потребления ресурсов каждым процессом.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Руссинович, М. Внутреннее устройство Windows. 7-е изд. / М. Руссинович, Д. Соломон, А. Ионеску, П. Йосифович. – СПб. : Питер, 2018. – 944 с. – (Серия «Классика computer science»). – ISBN 978-5-4461-0663-9.
2. Васильев, А. Н. Программирование на C# для начинающих. Основные сведения / А. Н. Васильев. – М. : Литрес, 2018. – 346 с. – ISBN 978-5-04-118420-9.
3. Microsoft. Документация по C# [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/dotnet/csharp/>. – Дата доступа: 29.10.2025.
4. Microsoft. Windows Presentation Foundation (WPF) documentation [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/dotnet/desktop/wpf/>. – Дата доступа: 29.10.2025.
5. Microsoft. Обзор шаблона Model-View-ViewModel [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/windows/communitytoolkit/mvvm/introduction>. – Дата доступа: 31.10.2025.
6. LiveCharts2 [Электронный ресурс] : документация. – Режим доступа: <https://livecharts.dev/docs/wpf/2.0.0-rc6.1/gallery>. – Дата доступа: 07.11.2025.
7. Microsoft. Windows Management Instrumentation (WMI) [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/windows/win32/wmisdk/wmi-start-page>. – Дата доступа: 10.11.2025.
8. Microsoft. Win32 Classes [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/windows/win32/cimwin32prov/win32-provider>. – Дата доступа: 10.11.2025.
9. Microsoft. Монитор производительности Windows [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/windows/win32/perfctrs/performance-counters-portal>. – Дата доступа: 10.11.2025.
10. Microsoft. Пространство имен System.Diagnostics [Электронный ресурс]. – Режим доступа: <https://docs.microsoft.com/dotnet/api/system.diagnostics>. – Дата доступа: 10.11.2025.

11. Microsoft. System.Security.Cryptography.X509Certificates
Пространство имен [Электронный ресурс]. – Режим доступа:
<https://learn.microsoft.com/ru-ru/dotnet/api/system.security.cryptography.x509certificates?view=net-9.0>. – Дата доступа: 16.11.2025.
12. Process Explorer [Электронный ресурс] : утилита от Sysinternals. –
Режим доступа: <https://learn.microsoft.com/en-us/sysinternals/downloads/process-explorer>. – Дата доступа: 01.11.2025.
13. RAMMap [Электронный ресурс] : утилита от Sysinternals. – Режим
доступа: <https://learn.microsoft.com/en-us/sysinternals/downloads/rammap>. – Дата доступа: 01.11.2025.
14. Metanit. WPF [Электронный ресурс]. – Режим доступа:
<https://metanit.com/sharp/wpf/>. – Дата доступа: 29.10.2025.